UNIEGO
Centro Universitário
Evangélico de Goianésia

CURSO DE ENGENHARIA DE SOFTWARE

Disciplina de Laboratório de Programação

Aula 12: Arquivos em Algoritmos

Prof. Me. Ivandro José de Freitas Rocha

Goianésia – 2026

OBJETIVO

Compreender o arquivo como estrutura de armazenamento em memória secundária, identificando seus elementos de organização e aplicando as operações primitivas na construção de algoritmos de manipulação de registros em arquivos sequenciais.

CONTEÚDOS

- Registro, tabela e arquivo: memória principal e memória secundária.
- Anatomia do arquivo: nome, caminho, extensão, ponteiro de posição e fim de arquivo (FDA).
- Operações primitivas: abrir com modo, ler, escrever, posicionar, testar fim e fechar.
- Notação algorítmica da disciplina.
- Aplicação: agenda telefônica (inserção, consulta, alteração e eliminação).

REFERÊNCIA

PIVA JR., D.; ENGELBRECHT, A. de M.; NAKAMITI, G. S.; BIANCHI, F. **Algoritmos e Programação de Computadores**. 2. ed. São Paulo: GEN/LTC, 2019 (Cap. 15).

ONDE OS DADOS VIVEM?

Quando um programa é encerrado, todas as variáveis e estruturas armazenadas na **memória principal (RAM)** são perdidas. Os dados simplesmente **deixam de existir**. Mas e quando precisamos que as informações *persistam* entre execuções?

🧠 Memória Principal

Rápida, temporária. Os dados vivem apenas durante a execução do programa.

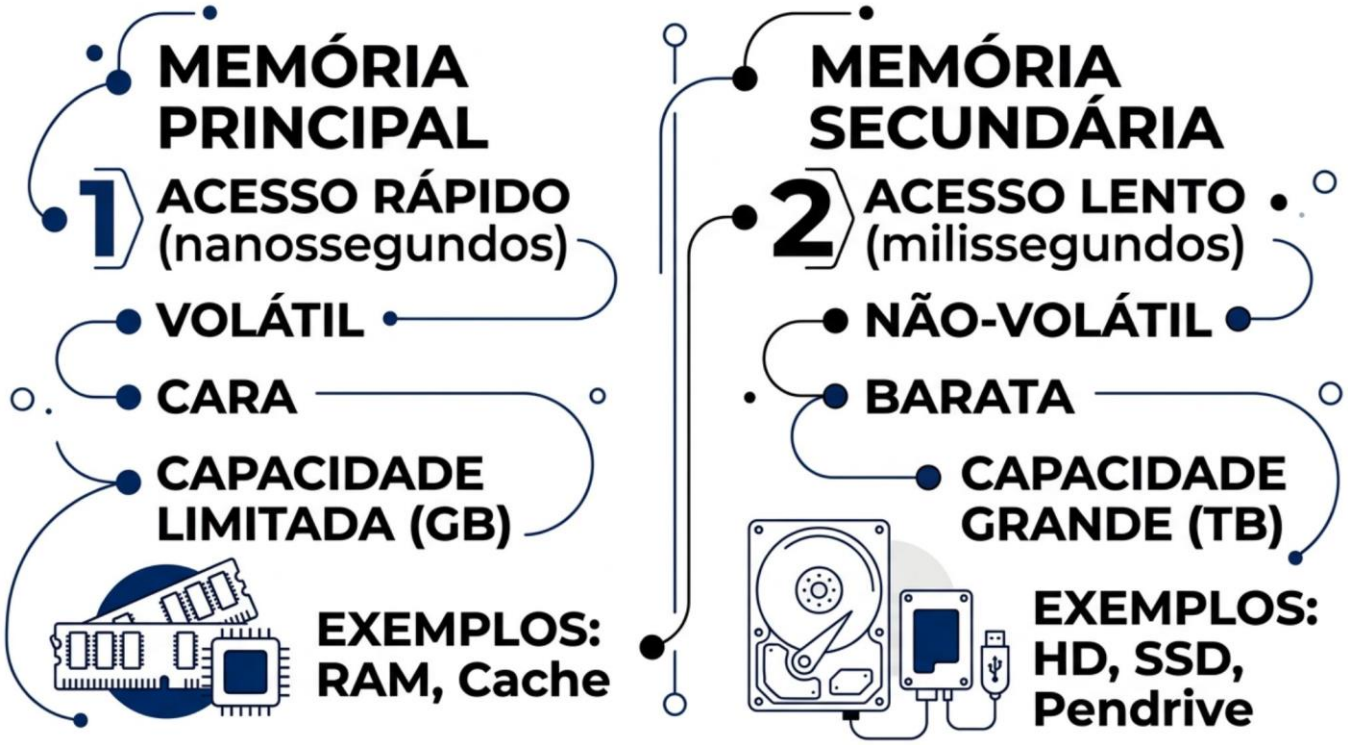
💾 Memória Secundária

Persistente. Os dados sobrevivem ao encerramento do programa, estão em disco.

📂 Arquivos

São o mecanismo que permite salvar e recuperar dados da memória secundária entre execuções.

MEMÓRIA PRINCIPAL × MEMÓRIA SECUNDÁRIA



Tabela

Conjunto de registros armazenados na **memória principal**. Existe apenas enquanto o programa está em execução.

Arquivo

Conjunto de registros armazenados na **memória secundária**. Persiste após o encerramento do programa.

REGISTRO = FICHA CATALOGRÁFICA

Um **registro** é uma estrutura composta por campos relacionados que descrevem uma entidade do mundo real. Assim como uma **ficha catalográfica** em uma biblioteca reúne todas as informações sobre um livro, um registro agrupa campos de tipos diferentes em uma única unidade lógica.

- Cada campo do registro armazena um atributo específico da entidade: texto, número ou valor lógico.

EXEMPLO: Registro de Livro

```
declare Tlivro registro(
    codigo, ano numérico
    titulo, autor, assuntos, editora literal
}
```

Ficha Catalográfica

Código:	Ano:
Título:	
Autor:	
Assuntos:	
Editora:	

ARQUIVO = FICHÁRIO

Um **arquivo** é uma sequência de registros D_1, D_2, ..., D_n armazenada em memória secundária e terminada por uma marca especial chamada **EOF (End Of File),** fim de arquivo. Assim como um fichário físico guarda fichas catalográficas em gavetas, um arquivo digital guarda registros em disco.

Sequência de Registros

$D_1 \rightarrow D_2 \rightarrow D_3 \rightarrow ... \rightarrow D_n \rightarrow$ **EOF**

Definição do Tipo

```
declare <identif_arquivo> arquivo composto <identif_registro>
```

Persistência

Os dados existem mesmo após o término do programa.



SISTEMA DE GERENCIAMENTO DE UMA BIBLIOTECA

Definição do registro que compõe o arquivo de uma biblioteca

```
declare tipo_livro registro (
      codigo, ano numérico,
      titulo, autor, assunto, editora literal
)
// Definição do arquivo
declare tipo_arqlivro arquivo composto tipo_livro
// Declaração de variáveis
declare ficha tipo_livro          // variável de registro
declare biblos tipo_arqlivro      // variável de arquivo
```

LEITOR × FUNCIONÁRIO

Diante de um fichário (arquivo), existem dois perfis distintos de usuário, cada um com permissões e ações diferentes. Essa distinção mapeia diretamente os **modos de abertura** do arquivo.

Leitor

Apenas **consulta** informações. Não altera, não insere, não remove. Abre o arquivo em modo `LEITURA`.

Funcionário

Manipula o arquivo de diferentes formas: inserir novos registros, modificar dados existentes e remover registros. Usa modos `ESCRITA`, `ACRESCIMO` ou `LEITURA_ESCRITA`.

A GAVETA COMO METÁFORA DAS OPERAÇÕES

Toda operação sobre um arquivo segue a mesma lógica do fichário físico: é preciso **abrir a gaveta** antes de qualquer ação e **fechá-la** ao terminar. Esse protocolo garante integridade dos dados.

1. Abrir a gaveta

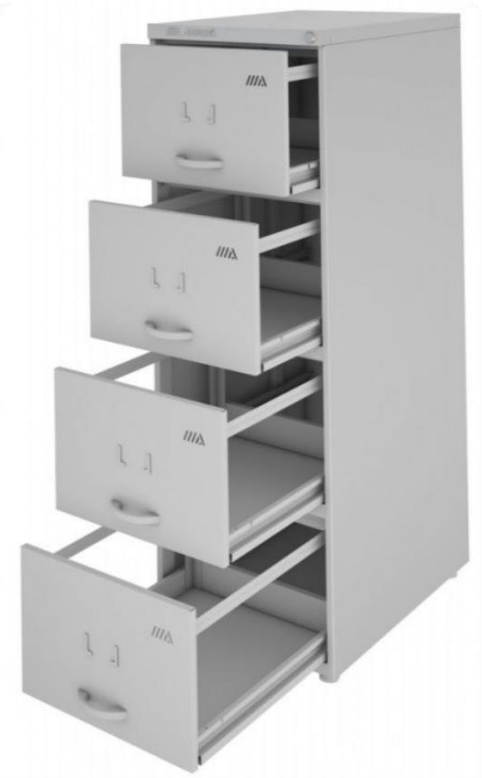
Tornar o arquivo acessível ao programa

2. Executar a operação

Consultar, inserir, alterar ou eliminar

3. Fechar a gaveta

Garantir que os dados foram gravados e protegidos



AS QUATRO ROTINAS DO FICHÁRIO



Consultar
1. Abrir o arquivo
2. Localizar a ficha desejada
3. Copiar informações
4. Fechar o arquivo



Acrescentar
1. Abrir o arquivo
2. Posicionar no final
3. Guardar a ficha nova
4. Fechar o arquivo



Modificar
1. Abrir o arquivo
2. Localizar a ficha
3. Alterar os dados
4. Gravar na mesma posição
5. Fechar o arquivo



Eliminar
1. Abrir o arquivo
2. Localizar a ficha
3. Reescrever o arquivo sem o registro
4. Fechar o arquivo

ABRIR ARQUIVO

Não podemos obter uma informação contida em um arquivo sem antes abri-lo.

```
abra <identif_arquivo>
```

no qual:

`identif_arquivo`: representa o identificador da variável arquivo previamente definida.

EXEMPLO: `abra biblos`

Sempre que o comando ABRA é executado, a **posição corrente** do arquivo é o **primeiro registro**.

PONTEIRO DE POSIÇÃO

É um marcador interno que indica *qual registro será lido ou gravado na próxima operação*. Ele é gerenciado automaticamente pelo Sistema, cada operação de `AVANCE` o avança para o registro seguinte.

avance <identif_arquivo>

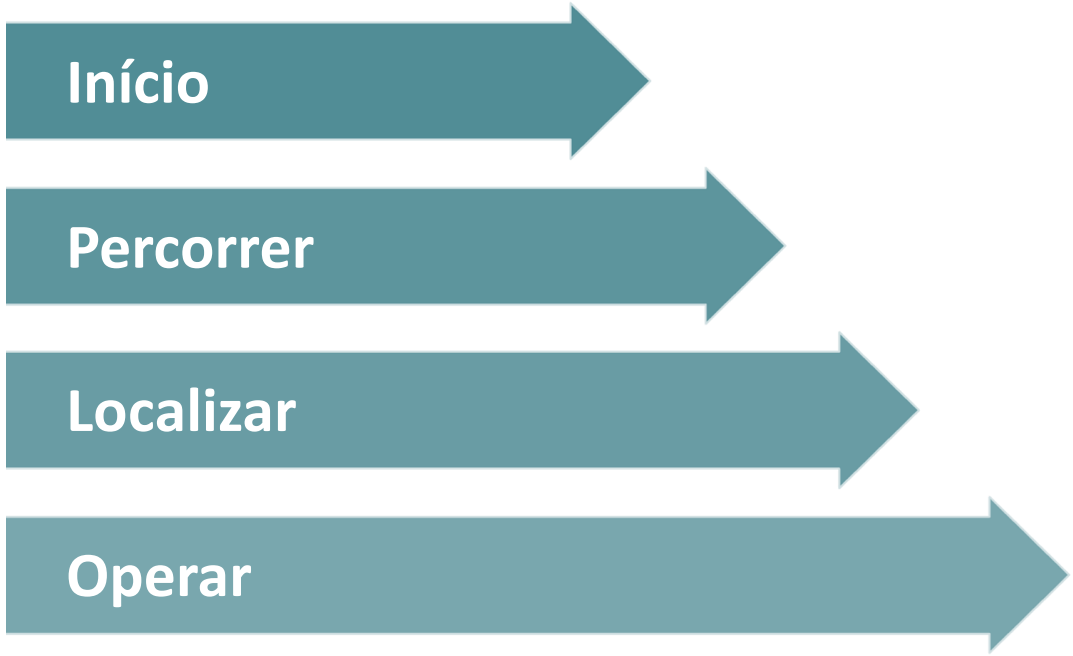
O comando coloca o arquivo na posição consecutiva ou seja, no próximo registro. Se utilizado repetidas vezes permite percorrer o arquivo passando por uma série consecutiva de registros.

EXEMPLO: `avance biblos`

Ao utilizar o comando ABRA o ponteiro é posicionado no **primeiro registro** do arquivo.

ORGANIZAÇÃO SEQUENCIAL

Na organização sequencial, os registros são armazenados **continuamente, um após o outro**, na ordem em que foram gravados. Para acessar um registro específico, é necessário percorrer todos os registros que o antecedem, não há acesso direto.



Esse modelo é simples de implementar e eficiente para operações que percorrem o arquivo inteiro, como relatórios ou intercalação. O custo é o tempo de busca proporcional ao tamanho do arquivo.

COPIANDO UM REGISTRO DO ARQUIVO

Um arquivo não deve ser consumido (no sentido de retirar informação dele) e sim consultado. Para tal, é necessário copiar o conteúdo que nos interessa em algum lugar.

```
copie <identif_arquivo>, <identif_registro>
```

no qual:

`identif_arquivo`: identificador de variável arquivo previamente definida;

`identif_registro`: identificador de variável registro de formato igual àquele que compõe o arquivo.

EXEMPLO: `copie biblos, ficha`

GRAVANDO UM REGISTRO NO ARQUIVO

Para guardar um registro no arquivo é necessário que este tenha a mesma estruturação de campos que os registros já armazenados.

```
guarde <identif_arquivo>, <identif_registro>
```

EXEMPLO: guarde biblos, ficha

- O comando GUARDE armazena o registro na **posição corrente** do arquivo.

ELIMINANDO UM REGISTRO DO ARQUIVO

Para eliminar informações indesejadas.

```
elimine <id_arquivo>
```

EXEMPLO: elimine biblos

OBSERVAÇÃO: o comando ELIMINE não apaga o registro, ele reescreve o arquivo sem o registro.

O comando ELIMINA reescreve o registro na **posição corrente** do arquivo.

FECHAR ARQUIVO

Não é recomendável manter um arquivo aberto depois de usá-lo, pois isso deixaria seu conteúdo exposto a agentes externos.

```
fecha <Identif_arquivo>
```

EXEMPLO: `fecha biblos`

FIM DE ARQUIVO (FDA)

Por que o FDA existe?

O sistema precisa saber onde o arquivo termina para não ler dados inválidos ou lixo de memória além do último registro gravado.

```
fda <Identif_arquivo>
```

A função FDA retorna VERDADEIRO quando o ponteiro atingiu o fim do arquivo, e FALSO caso contrário.

Uso em Laços de Repetição

```
abra biblos
enquanto não fda biblos faça
    avança biblos
fecha biblos
```

DEFINIÇÃO DO REGISTRO

Desenvolva um sistema que gerencie uma agenda de telefones, para isso crie os seguintes módulos: Inserção, consulta, alteração e exclusão.

```
declare  tipo_pessoa registro(
    nome literal,
    fone numérico
)
declare  tipo_arqpessoa arquivo composto tipo_pessoa
```

MODULO DE INSERÇÃO

O modulo de inserção irá criar um novo telefone em um arquivo sequencial.

```
01.  sub-rotina INSERÇÃO
02.     declare aux tipo_pessoa
03.             agenda tipo_arqpessoa
04.     abra agenda
05.     repita
06.        avance agenda
07.     até fda agenda
08.     leia aux.nome
09.     leia aux.fone
10.     guarda agenda, aux
11.     fecha agenda
12.  fim sub-rotina
```

MODULO DE CONSULTA

Como não se conhece a posição da informação procurada no arquivo, vasculha-se o arquivo partir do início. Se o final de arquivo for alcançado e a informação não for encontrada, conclui-se que esta não existe.

```
01.  sub-rotina CONSULTA
02.     declare aux tipo_pessoa
03.                 agenda tipo_arqpessoa
04.                 nome_procurado literal
05.     abra agenda
06.     leia nome_procurado
07.     repita
08.        copie agenda, aux
09.        avance agenda
10.     até aux.nome = nome_procurado ou fda agenda
11.     se aux.nome = nome_procurado
12.        então escreva aux.fone
13.        senão escreva "telefone não registrado"
14.     fecha agenda
15.  fim sub-rotina
```

Observações: Não deve-se, apenas, avançar pelos registros para verificar a coincidência do campo nome, é preciso copiar para um registro auxiliar cada um dos registros armazenados.

MODULO DE ALTERAÇÃO

Alguém cadastrado muda o telefone, temos então que localizar seu registro correspondente, copiar seu conteúdo a um registro auxiliar e então alterar o campo fone. Depois basta gravar as informações atualizadas na mesma posição em que se encontrava antes, ou seja, gravar o registro atualizado **por cima** do antigo.

```
01.  sub-rotina ALTERAÇÃO
02.     declare aux pessoa
03.             agenda pessoal
04.             nome_procurado literal
05.     abra agenda
06.     leia nome_procurado
07.     copie agenda, aux
08.     enquanto aux.nome ≠ nome_procurado
                  e (não fda agenda) faça
09.     início
10.       avance agenda
11.       copie agenda, aux
12.     fim
13.   se  aux.nome = nome_procurado
14.     então início
15.             escreva aux.fone
16.             escreva "Novo telefone"
17.             leia aux.fone
18.             guarde agenda, aux
19.           fim
20.     senão escreva "telefone não registrado"
21.   fecha agenda
22. fim sub-rotina
```

OBSERVAÇÃO: não avançar a posição corrente depois de ter encontrado o registro, pois após atualização, regravá-lo na mesma posição.

MODULO DE EXCLUSÃO

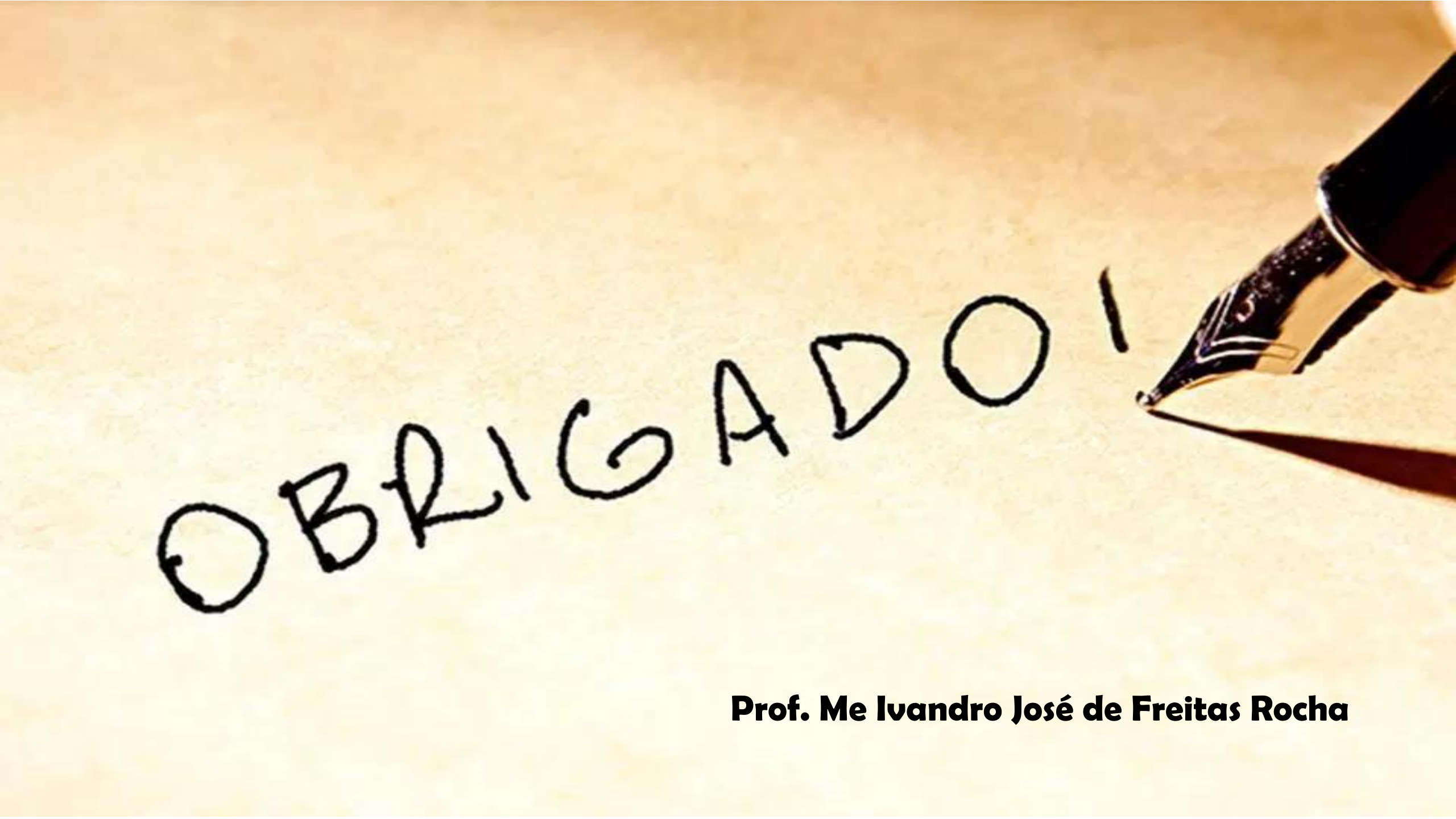
Se quisermos excluir algum telefone arquivado devemos primeiro localizar o registro para então eliminá-lo após uma confirmação.

```
01. sub-rotina ELIMINAÇÃO
02.   declare aux pessoa
03.           agenda pessoal
04.           nome_procurado,
05.           confirmação literal
06.   abra agenda
07.   leia nome_procurado
08.   copie agenda, aux
09.   enquanto aux.nome ≠ nome_procurado
                e não ( fda agenda ) faça
10.   início
11.     avance agenda
12.     copie agenda, aux
13.   fim
14.   se aux.nome = nome_procurado
15.     então início
16.             escreva aux.nome, aux.fone
17.             escreva "Excluir (s/n)?"
18.             leia confirmação
19.             se confirmação = "s"
20.                 então elimine agenda
21.         fim
22.     senão escreva "telefone não registrado"
23.   fecha agenda
24. fim algoritmo
```

OBSERVAÇÃO: não avançar a posição corrente depois de ter encontrado o registro, pois o registro da posição corrente que será eliminado.



ivandro.rocha@uniego.edu.br



Prof. Me Ivandro José de Freitas Rocha